
CONVERTING 2D TO 3D IMAGE

Application Development-II Report Submitted
In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Electronics and Communication Engineering

by

J.Anil Kumar	21N31A0491
K.Abhinay Reddy	21N31A0499
G.Srinivasa Reddy	21N31A0473

Under the esteemed guidance of

Mrs.Dr.P.Vanitha



Department of Electronics and Communication Engineering

Malla Reddy College of Engineering & Technology

(Autonomous Institution-UGC, Govt. of India)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade)

Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100

website: www.mrcet.ac.in

2021-2025



Malla Reddy College of Engineering & Technology (Autonomous Institution-UGC, Govt. of India)

(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA & NAAC with 'A' Grade)

Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100

website: www.mrcet.ac.in

CERTIFICATE

This is to certify that this is the bonafide record of the **Application Development-II** entitled **“COVERTING 2D TO 3D IMAGE”**, submitted by **J.Anil Kumar (21N31A0491)**, **K.Abhinay Reddy (21N31A0499)** and **G.Srinivasa Reddy (21N31A0473)** of B.Tech in the partial fulfillment of the requirements for the degree of Bachelor of Technology in Electronics and Communication Engineering, Department of Electronics and Communication Engineering during the year 2023-2024. The results embodied in this Application Development-II report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide

Mrs.Dr.P.Vanitha

Associate Professor

Head of the Department

Dr. K. Mallikarjuna Lingam

Professor & HOD

External Examiner

DECLARATION

We hereby declare that the Application Development-II titled “**CONVERTING 2D TO 3D IMAGE**” submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering is a result of original research carried-out in this thesis. It is further declared that the Application Development-II report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

J. Anil kumar - 21N31A0491

K. Abhinay Reddy- 21N31A0499

G. Srinivasa Reddy-21N31A0473

ACKNOWLEDGMENTS

First and foremost, we would like to thank God for giving strength to complete this work. We feel ourselves honoured and privileged to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) and our Director Dr. VSK Reddy, Principal Dr. S. Srinivasa Rao who gave us the opportunity to have experience in engineering and profound technical knowledge.

We express our heartiest thanks to Prof P. Sanjeeva Reddy, Director of the Department of Electronics and Communication Engineering and Dr.K. Mallikarjuna Lingam, Head of the Department of Electronics and communication engineering for encouraging us in every aspect of our project and helping us realize our full potential.

We would like to thank our Project Coordinator Mrs.Dr.P.Vanitha, Associate Professor, Department of Electronics and Communication Engineering for her valuable guidance and encouragement during our project work.

We would like to thank our internal guide Mrs.Dr.P.Vanitha, Designation, Department of Electronics and Communication Engineering for her regular guidance and constant encouragement. We are extremely grateful to her valuable suggestions and unflinching cooperation throughout project work.

We would like to thank Dr. T. Venugopal, Dean of Academics, who in spite of being busy with his duties took time to guide and keep us on the correct path.

We would also like to thank all the supporting staff of the Department of Electronics and Communication Engineering and all other departments who have been helpful directly or indirectly in making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completion of our project that gave us strength to do our project.

With regards and gratitude,

**J.Anil Kumar (21N31A0491)
K.Abhinay Reddy (21N31A0499)
G.Srinivasa Reddy (21N31A0473)**

CONTENTS

CONTENTS	Page No
Cover Page	i
Certificate	ii
Declaration	iii
Acknowledgements	iv
List of Figures	vi
Abstract	1
Chapter-1 Introduction	2-4
1.1 Motivation of proposed system	3
1.2 Existing System	3
1.3 Proposed system	4
Chapter-2 System Requirements	5-6
2.1 Software requirements	5
2.2 Technologies Used	5
2.3 Principle of Stereoscopic Image	5-6
Chapter-3 System Design	7-9
3.1 Libraries used in system	7-8
3.2 UML diagram	8-9
Chapter-4 Implementation of Proposed system	10-13
4.1 Algorithm of 3D image	10
4.2 System Implementation	10-13
Chapter-5 Results and Discussions	14-16
5.1 Description	14
5.2 Description with Results	14-16
Chapter-6 Conclusions and Future Scope	17
6.1 Advantages and Applications	17
6.2 Conclusion	17
6.3 Future Scope	17
References	18
Appendix A: Source Code	19-22

LIST OF FIGURES

Fig. No.	Name of the figure	Pg No.
2.1	The Principle of Stereoscopic Image	6
3.1	UML Diagram	8
4.1	Block Diagram of Converting 2D to 3D Image	11
4.2	Reconstruction of Input Image	13
5.1	Output of Input Image 1	14
5.2	Output of Input Image 2	15
5.3	Output of Input Image 3	15

ABSTRACT

In this post we will explore a recent attempt of extending DL to the Single image 3D reconstruction task, one of the most important and profound challenge in the field of 3D computer graphics. A three-dimensional video enhances the illusion of depth perception by allowing us to see an object from different angles. Images in 3D are used in movies and games for animation. Additionally, these are used for educational purposes, medicine, engineering, and technology prototyping and architecture. This project involves the generation of 3D videos from 2D images using Python. Here, we use Multi-Level Pixel-Aligned Implicit Function for High-Resolution 3D Human Digitization (pifuHD) to get the 3Dimensional image. A single image is only a projection of 3D object into a 2D plane, so some data from the higher dimension space must be lost in the lower dimension representation. Therefore, from a single-view 2D image, there will never be enough data construct its 3D component. A method to create the 3D perception from a single 2D image therefore requires prior knowledge of the 3D shape in itself.

Keywords: Extension DL, Dimension, illusion of depth, Technology prototyping.

CHAPTER 1

INTRODUCTION

The two dimensions depicted are length and width and the objects on the picture are flat. Examples of such pictures are ancient Egyptian wall paintings or images from video games before the Play Station era, where visual artists did not want, or could not, give a realistic representation of space when converting from 2d image to 3d model, there are several options to choose from – whether or not you have experience in going from 2D to 3D. 3D means three-dimensional, or having three dimensions. The three dimensions are often called length (or depth), width (or breadth), and height.

The convenience of 3D-capable hardware today, such as TVs, Blu-Ray players, gaming consoles, and smart phones is not yet matched by 3D content production. Today there exists an urgent need to convert the existing 2D content to 3D. A typical 2D-to-3D conversion process consists of two steps: depth estimation for a given 2D image and depth based rendering of a new image in order to form a stereo pair. While the rendering step is well understood, the challenge is in estimating depth from a single image. Therefore, throughout the focus is on depth recovery.

There are two basic approaches, semi-automatic and automatic methods. In the former case a skilled operator assigns depth to various parts of an image. Based on this sparse depth assignment, a computer algorithm estimates dense depth over the entire image or sequence. In the case of automatic methods, no operator involvement is needed and a computer algorithm automatically estimates the depth for a single image. Recently, machine-learning-inspired methods have been proposed to automatically estimate the depth map of a single monocular image by applying image parsing.

Two types of methods are proposed. The first one is based on learning a point mapping from local image/ attributes, such as color, spatial position, and motion at each pixel, to scene depth at that pixel using a regression type idea. The second one is based on globally estimating the entire depth map of a query image directly from a repository of 3D images (image + depth pairs or stereo pairs) using a nearest-neighbor regression type idea.

1.1 MOTIVATION OF PROPOSED SYSTEM

The aim in this project is to use a turntable or a robot using which we can obtain photographs of a given object from known angles. We aim to reconstruct and model these 2D images to obtain the 3D form of the object, which will be developed and exhibited in OPENGL. Thus our project does not limit itself to giving the user the freedom of viewing the object at any angle and in any orientation but provides us with the complete model in space. 3D reconstruction and modeling are used in many fields like Virtual Reality, recognizing and manipulating objects etc...

1.2 EXISTING SYSTEM

The convenience of 3D-capable hardware today, such as TVs, Blu-Ray players, gaming consoles, and smart phones, is not yet matched by 3D content production. Today there exists an urgent need to convert the existing 2D content to 3D. A typical 2D-to-3D conversion process consists of two steps: depth estimation for a given 2D image and depth based rendering of a new image in order to form a stereo pair. While the rendering step is well understood, the challenge is in estimating depth from a single image. Therefore, throughout the focus is on depth recovery.

Disadvantages of Existing System:

It requires a human operator when the Grayscale image used because these images contain shades of black and white and contain only single channel.

- Doesn't work perfectly on images with lots of details or text.
- Sometimes it only renders a portion of the original file. Not available for Mac users.
- Doesn't work with blurry photos or photos of blank walls.
- Expensive.

1.3 PROPOSED SYSTEM

The proposed methods carry the big data philosophy of machine learning. They apply to arbitrary scenes and require no manual explanation. Two types of methods are proposed. The first one is based on learning a point mapping from local image/ attributes, such as color, spatial position, and motion at each pixel, to scene-depth at that pixel using a regression type idea. The second one is based on globally estimating the entire depth map of a query image directly from a repository of 3D images (image + depth pairs or stereo pairs) using a nearest-neighbor regression type idea. It introduces local method and evaluates the qualitative performance and the computational efficiency of both the local and global methods. The improved quality of the depth maps produced by the global method relative to state-of-the-art methods together with up to 4 orders of magnitude reduction in computational effort and weakness of the methods are also demonstrated. We used the language of python and running in the pifuHD to get the 3D visual. To check the accuracy of proposed method we use Support Vector Machine (SVM). It is one of the most popular supervised Learning algorithms. SVM provides significantly better classification accuracy and classification speed.

Advantages of proposed system:

- The proposed methods carry the big data philosophy of machine learning. They apply to arbitrary scenes and require no manual explanation.
- It can be used for converting group of 2D images into 3D easily.
- Proposed system has more accuracy compared to KNN, Logistic Regression.
- It is not much expensive compared to existing method.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 SOFTWARE REQUIREMENTS

- Windows X64 Operating system
- Python language
- Spyder software or any python compiler
- High processing machine for large dataset
- Tools such as Python and libraries such as Json, skimage, Numpy, OpenCV will be utilized for this process.

2.2 TECHNOLOGIES USED

Stereoscopic images are a new concept in media where a person can feel reality because it is similar to actuality of real world. A realistic stereoscopic experience is achieved using, by a stereoscopic camera or created by an image editor. However, creation of a stereoscopic image using image editor is very costly and time consuming. An additional difficulty is the need to convert the massive existing 2D image contents to stereoscopic images. Stereoscopic image conversion techniques produce a stereoscopic image using 2D images created by a single mono camera. A conventional conversion method for stereoscopic images is MTD (Modified Time Difference). Using MTD, SANYO developed a 2D to 3D conversion TV in 1993 and Okino developed the first commercial 2D to 3D image conversion TV. MTD creates a stereoscopic by making the current image into the left image and previous image into the right image. In this paper, stereoscopic image conversion method using a single frame of a 2D image. A depth map based on the vertical position information is then created and converted into a stereoscopic image through parallax processing.

2.3 PRINCIPLE OF STEREOSCOPIC IMAGE

A person sees a slightly different image through each eye as each eye is positioned away from other with a regular distance. Such different images are then fused into a single stereoscopic image in the brain. This is known as binocular disparity. Most stereoscopic

images are created using binocular disparity, as shown in Figure 2.1.

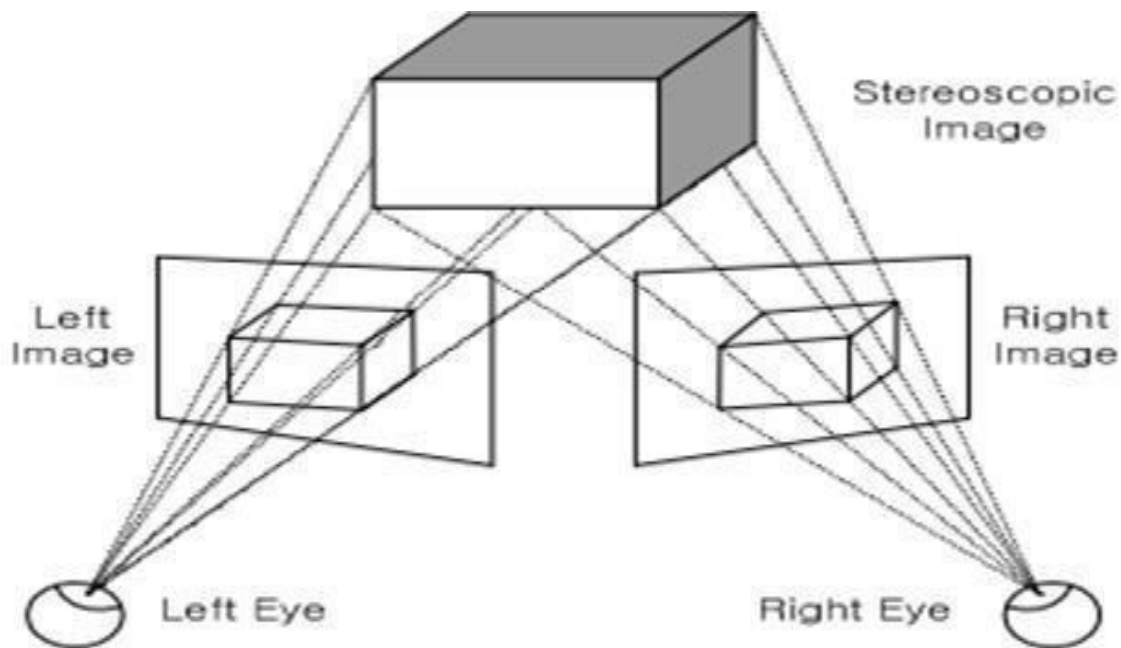


Fig. 2.1. The Principle of Stereoscopic Image

When a person views their surroundings stereoscopically, many cues of depth perception are applied. These are greatly classified into two categories, binocular cues and monocular cues. Binocular cues include convergence and binocular disparity. Convergence occurs when human eyes rotate toward each other to set a visual point on the object when a human gazes an object. The perception of depth by convergence has a great effect in a short distance. Monocular cues include relative size, aerial perspective, textural gradient, light and shade and interposition. These constitute a motion parallax obtained by the motion difference of object according to the change of relative position with observer.

CHAPTER 3

SYSTEM DESIGN

Design is concerned with identifying software components specifying relationships among components.

To covert 2D image to 3D image :

1. Install Microsoft 3D Builder.
2. Uploading an image and converting it to 3D.
3. Which conversion method to choose.
4. Level selection and smoothing.
5. Scale your image to the size you want.
6. Save the image.

The design activity is often divided into two separate phase system design and detail design. System design, which is sometimes also called top-level design, aims to identify the modules that should be in the system, the specifications of these modules, and how they interact with each other to produce the desired results.

During detailed design the internal logic of the modules specified in system design is decided. During this phase further details of the data structures and algorithmic design of each of the modules is specified. The logic of a module is usually specified in a high-level design description language, which is independent of the target language in which the software will eventually be Implemented.

3.1 LIBRARIES USED IN SYSTEM:

OpenCV : The library consists of around 2000+ optimized algorithms that are useful for computer vision and machine learning. It is useful to convert images from one color space to another.

NumPy : NumPy is a library can also perform simple image techniques, such as flipping images, extracting features and analyzing them. Images can be represented by numpy multi-dimensional arrays and so their type is NdArrays.

SKIMAGE : It is a open source library used for Image processing. It makes use of machine learning with built in functions and can perform complex operations on images with just few functions. It works with numpy and arrays.

JSON : It benchmarks as the fastest python libraries for JSON and is more correct than the standard JSON library. It is a storing perfect data.

3.2 UML DIAGRAM

By these we want to show the things which are associated with our model and the relationship between those things. It shows how elements are associated with each other and this association describes the functionality of our application.

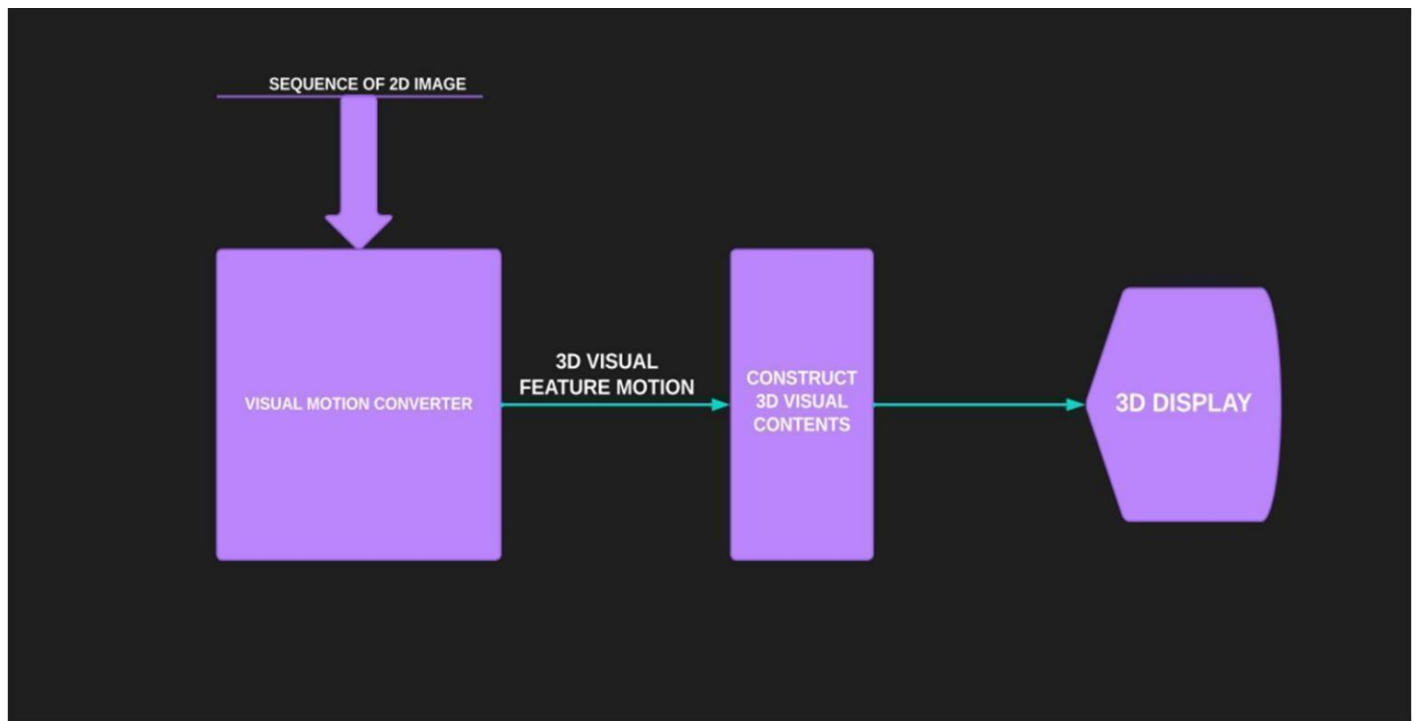


Fig.3.1 UML DIAGRAM

From the above figure 3.1 firstly, we collect the images as sequence of 2D images then moves to visual motion converter it is the process of recording the movement of objects or people and by this it carries the 3D visual feature motion and convert to the construct of 3D visual contents, when it constructs the 3D visual then the 3D image is displayed. Therefore, the 3D visualization is the process of creating and displaying digital content. It will be improved collaboration and greater productivity across architecture, engineering, medical, marketing and etc.

The Video Resolution for converting 2D Image to 3D Image using Python will depend on the specific requirements of the project. The resolution of the Input Image and desired output resolution of the 3D Image will consider. Input Resolutions will generally produce better quality 3D output.

CHAPTER 4

IMPLEMENTATION OF PROPOSED SYSTEM

4.1 ALGORITHM OF 3DIMAGE

In this algorithm low level image attributes namely color, location and motion (in case of video) at each pixel are to be considered. So, in this algorithm depth is assigned to a pixel based on its attributes. For the given 2D query input image, the 3D output is generated which uses the color transformation, location transformation and in case of video motion transformation at each pixel. And finally the outputs of all this transformations are getting combined to create the final 3D output with depth assigned at each pixel. A pivotal element in this approach is a point transformation used to compute depth from image attributes. This means, for a given 2D query image, if taking 'k' numbers of photos through camera. Then by referring the original image, considering each pixel in the other photos that by how much location it is moved from one place to another then how much color get changed and finally what about its motion at that pixel. So, for that 'k' images we will get that much of depth and by combining the image with its depth and creating final 3D image as output.

4.2 SYSTEM IMPLEMENTATION

Semiautomatic method of 2D to 3D image conversion requires human operator involvement for scribbling and error correction. But the algorithm which uses semiautomatic method of 2D to 3D image conversion is costly and speed of conversion is slow. So, in order to reduce operator involvement in the process and, therefore, lower the cost while speeding up the conversion, the automatic method will simplified algorithms is used. In this system simplified algorithm is implemented using automatic method for 2D to 3D image conversion

BLOCK DIAGRAM OF CONVERTING 2D TO 3D IMAGE

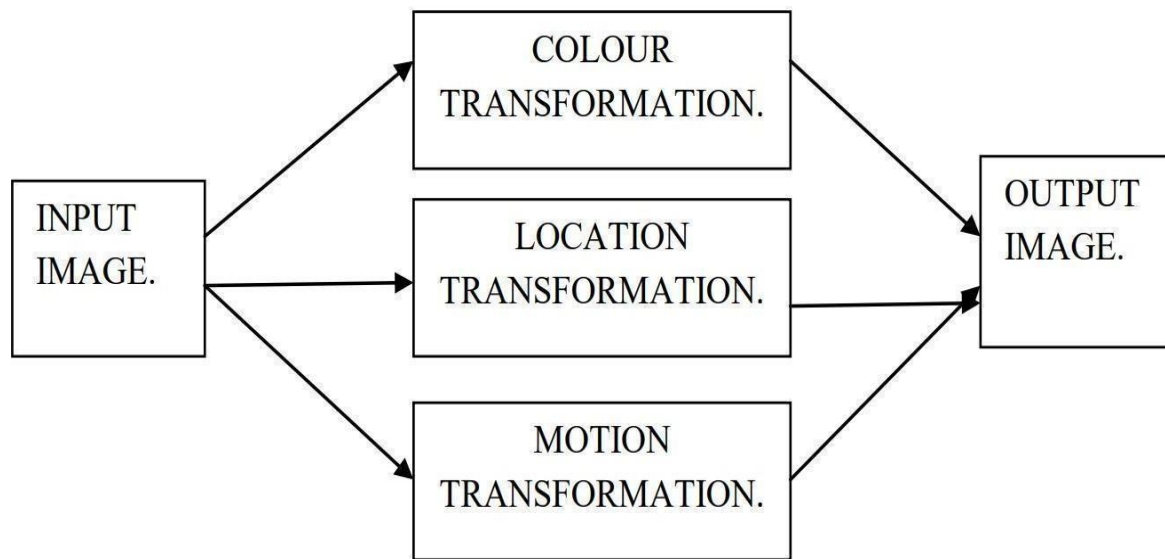


Fig. 4.1. Block diagram of Converting 2D to 3D Image

COLOUR TRANSFORMATION :-

$$r = R/(R+G+B),$$

$$g = G/(R+G+B),$$

$$b = B/(R+G+B),$$

$$r + g + b = 1$$

LOCATION AND MOTION TRANSFORMATION :-

$F(x, y)$ = Input image on which transformation function.

$G(x, y)$ = Output image or processed image.

$$P = [X, Y, Z]^T$$

It is a 3D point in the conventional camera reference frame.

The relative motion V between P and the camera can be described as:

$$V = -T - \omega * P$$

Where T and ω are the translational velocity vector and the angular velocity of the camera respectively.

$$\text{➤ } V_x = (T_z x - T_x f)/Z - \omega_y f + \omega_z y + (\omega_x xy)/f - (\omega_x x^2)/f$$

$$\text{➤ } V_y = (T_z x - T_y f)/Z - \omega_x f - \omega_z y - (\omega_y xy)/f + (\omega_x y^2)/f$$

where V_x and V_y are the components of motion field in x and y direction respectively.

Z is the depth of the corresponding 3D point; and the subscripts x , y and z indicate the component of the x-axis, y-axis and z-axis directions.

As shown in Figure 4.1, for the given 2D query input image, the 3D output is generated which uses the color transformation, motion transformation and location transformation at each pixel. And finally the outputs of all this transformations are getting combined to create the final 3D output with depth assigned at each pixel. A pivotal element in this approach is a point transformation used to compute depth from local low level image attributes.

Specification:

1. Image sensor: High quality 1/4 CMOS sensor
2. Effective Pixels: 480K pixels (Interpolated 8M pixels still image & 4M pixels video)
3. Video Resolution: Max. up to 2304 x 1728 pixels – 4MP
4. Image Resolution: Max. up to 3264 x 2448 pixels – 8MP
5. Frame Rate: 30 frames per second.

Pixel-Aligned Implicit Function

The goal of 3D human digitization can be achieved by estimating the occupancy of a dense 3D volume, which determines whether a point in 3D space is inside the human body or not. The goal of PIFu is to model a function, $f(X)$, which predicts the binary occupancy value for any given 3D position.

$$f(X, I) = \{ 1 \text{ if } X \text{ is inside mesh surface } 0 \text{ Otherwise } \}$$

where I is a single RGB image. PIFu models the function f via a neural network architecture that is trained in an end-to-end manner. Specifically, the function f first extracts a image feature embedding from the projected 2D location.

Then, it estimates the occupancy of the query 3D point X, and thus:

$$f(X, I) = g(\Phi(x, I), Z)$$

where $Z = X_z$ is the depth along the ray defined by the 2D projection x. 3D points along the same ray have exactly the same image features $\Phi(x, I)$ from the same projected location x, and thus the function g should focus on the varying input depth Z to disambiguate the occupancy of 3D points along the ray. In a Convolutional Neural Network (CNN) architecture is used for the 2D feature embedding function Φ and a Multilayer Perceptron

(MLP) for the function g . Here, in Convolutional Neural Network (CNN) takes input as an image volume for the RGB image.

Multi-Level Pixel-Aligned Implicit Function

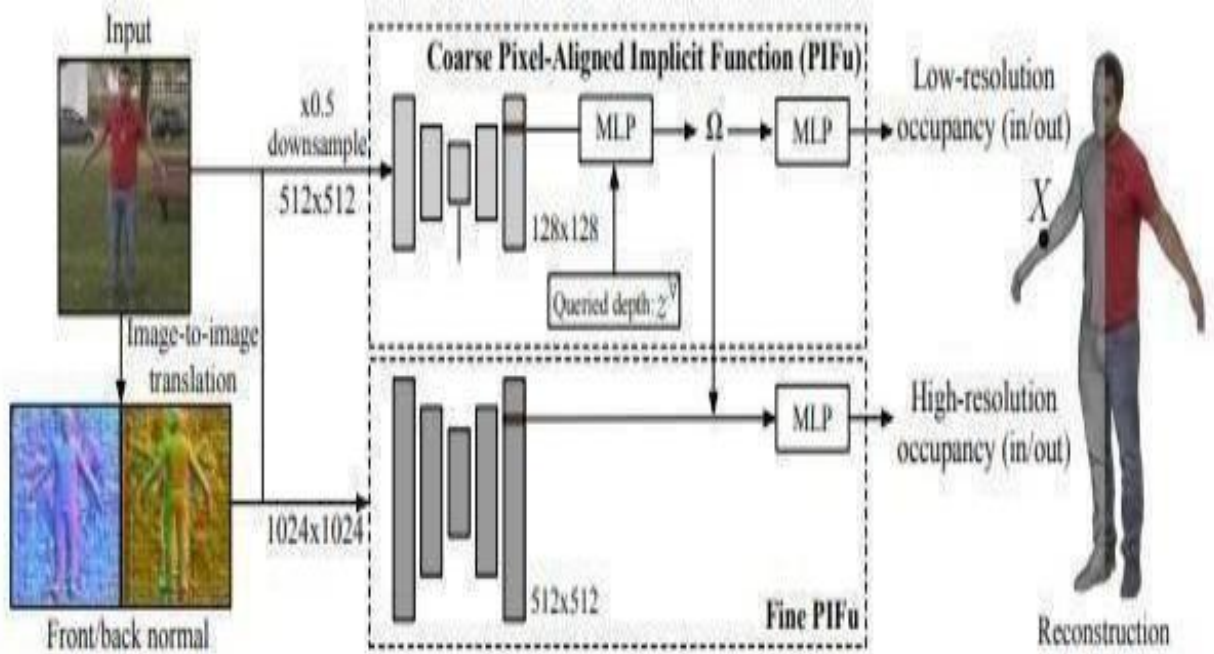


Fig. 4.2. Reconstruction of Input Image

A multi-level approach towards higher fidelity 3D human digitization that takes 1024×1024 resolution images as input. Our method is composed of two levels of PIFu modules: (1) a coarse level similar to PIFu focusing on integrating global geometric information by taking the down sampled 512×512 image as input, and producing backbone image features of 128×128 resolution, and (2) a fine level that focuses on adding more details by taking the original 1024×1024 resolution image as input, and producing backbone image features of 512×512 resolution. Here, after taking the Input image it goes to number of layers by using MLP and gives the resolution occupancy. The total process is about pre trained model in pifuHD.

CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 DESCRIPTION

2D to 3D video conversion (also called 2D to stereo 3D conversion and stereo conversion) is the process of transforming 2D ("flat") film to 3D form, which in almost all cases is stereo, so it is the process of creating imagery for each eye from one 2D image. In 3D includes Height, Width and Depth for the image converter. And also we can show this in the form of video presentation of 3D image.

5.2 DESCRIPTION WITH RESULTS

Sample1:

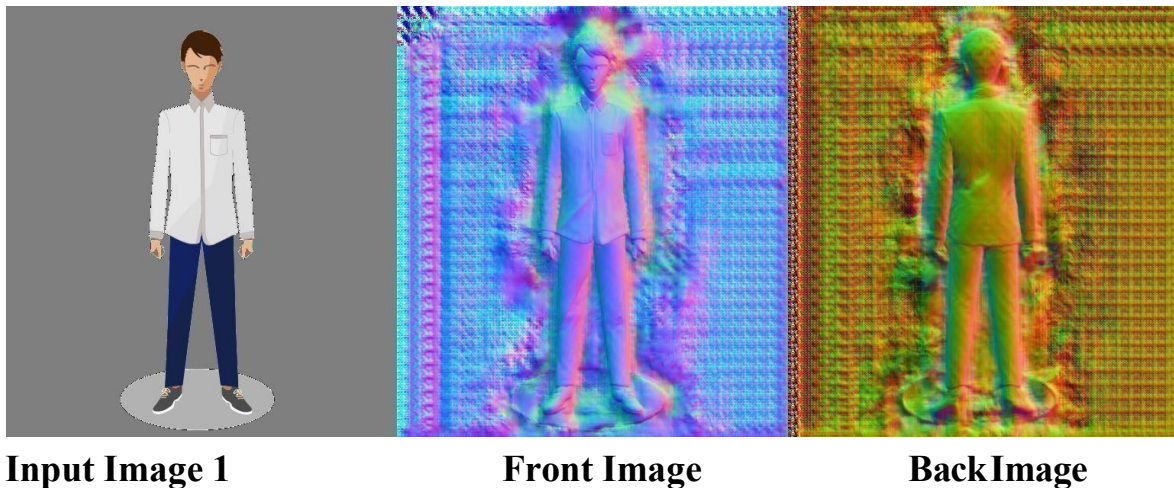


Fig. 5.1. Output of Input Image 1

Sample 2:



Fig. 5.2. Output of Input Image 2

Sample 3:



Fig. 5.3. Output of Input Image 3

The above figures displays the output as 3D image of the 2D image which we gave as input image. Here, in the Result it is displayed the Reconstructed geometry for the Input image which is in the process to get Output. And also there is a front and back of the images in the process of converting. As the side of the images can't displayed in the output of the image. But we can see the side views of the image in the video of the converting process of 2D to 3D image conversion. Here, we shown the different types of input images as sample wise and their respected output images too. And after getting of output images we also can display the video of the 3D visualization.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 ADVANTAGES AND APPLICATIONS

- Medical imaging
- Spatial Navigation
- Art and Animation
- Data Conversion
- Education and training
- Gaming
- Fashion Design

6.2 CONCLUSION

The amount of 3D content has greatly increased in recent years and there is more demand for 3D images in many scenarios. A single technique to convert the entire class of 2D images to 3D image models does not exist. It has been observed that machine learning is a new and promising research direction in 2D to 3D image conversion. And it is also helpful to explore the alternatives than to confine ourselves only in the conventional techniques based on depth maps. 2D content to 3D content conversion method which involves human operator in many algorithms are time consuming but having less efficiency whereas some algorithm which is not involving operator is less time consuming but having high efficiency. Many algorithms have been developed in the last few years to convert 2D to 3D images which are complex. PifuHD Algorithm which uses automatic method for conversion of 2D images to 3D images using point transformation technique is developed in this system. Our proposed system has more accuracy and less complexity.

6.3 FUTURE SCOPE

We observe that a lot of the 2D to 3D conversion algorithms are still in the research phase. The factors such as unsatisfactory quality and high complexity make it inadequate for real time use. A prior knowledge of the image content is needed. It is also claimed to be fully unsupervised and suitable for real-time applications.

REFERENCES

- [1] T. Alldieck, M. Magnor, B. L. Bhatnagar, C. Theobalt, and G. Pons-Moll. Learning to reconstruct people in clothing from a single RGB camera. In IEEE Conference on Computer Vision and Pattern Recognition, pages 1175–1186, 2019.
- [2] T. Alldieck, M. A. Magnor, W. Xu, C. Theobalt, and G. Pons-Moll. Video based reconstruction of 3d people models. In IEEE Conference on Computer Vision and Pattern Recognition, pages 8387–8397, 2018.
- [3] T. Alldieck, G. Pons-Moll, C. Theobalt, and M. Magnor. Tex2shape: Detailed full human body geometry from a single image. In The IEEE International Conference on Computer Vision (ICCV), October 2019.
- [4] R. Alp Guler, N. Neverova, and I. Kokkinos. Densepose: “Dense human pose estimation in the wild. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 7297–7306, 2018.
- [5] D. Anguelov, P. Srinivasan, D. Koller, S. Thrun, J. Rodgers, and J. Davis. SCAPE: shape completion and animation of people. ACM Transactions on Graphics, 24(3):408–416, 2005.
- [6] B. L. Bhatnagar, G. Tiwari, C. Theobalt, and G. Pons-Moll. Multi-garment net: Learning to dress 3d people from images. In Proceedings of the IEEE International Conference on Computer Vision, pages 5420–5430, 2019.
- [6] F. Bogo, A. Kanazawa, C. Lassner, P. Gehler, J. Romero, and M. J. Black. Keep it SMPL: Automatic estimation of 3D human pose and shape from a single image. In European Conference on Computer Vision, pages 561–578, 2016.
- [7] Janusz Konrad, Meng Wang, et al, “Learning-Based, Automatic 2D-to-3D Image and Video Conversion” IEEE Transactions on Image Processing, vol. 22, no. 9, September 2013.
- [9] The <http://iscte.pt/~pjln/MScProposals/MSc-Proposal-PauloNunes-2011-2D-to-3D-Video Conversion.pdf>
- [10] Media college, “what is 3D”, a post on MEDIA COLLEGE.com, February 1, 2009 available at: <http://www.mediacollege.com/3d>, retrieved on 14/08/2014.
- [11] http://www.javelin_tech.com/main/solutions/going_3d_benefits
- [12] Dr. Ir. E. A. Hendricks, Dr. Ir. P. A. Redert “Research Assignment Converting 2D to 3D”: A Survey Information and Communication Theory Group (ICT), December 2005.

APPENDIX A: SOURCE CODE FOR CONVERTING 2D IMAGE TO 3D IMAGE

Step-1

Install pytorch3d for Rendering

```
!pip install 'torch==1.6.0+cu101' -f https://download.pytorch.org/whl/torch\_stable.html  
!pip install 'torchvision==0.7.0+cu101' -f https://download.pytorch.org/whl/torch\_stable.html  
!pip install 'pytorch3d==0.2.5'
```

Step-2

Clone PIFuHD repository

```
!git clone https://github.com/facebookresearch/pifuhd
```

Step-3

Configure input data

```
cd /content/pifuhd/sample_images
```

If you want to upload your own picture, run the next cell. Otherwise, go to the next next cell.
Currently PNG, JPEG files are supported.

YOUR IMAGE SHOULD BE AROUND 60KB IN SIZE, OTHERWISE YOU WILL GET
AN

OUT OF MEMORY ERROR

```
from google.colab import files  
filename = list(files.upload().keys())[0]  
import os  
try:  
    image_path = '/content/pifuhd/sample_images/%s' % filename  
except:  
    image_path = '/content/pifuhd/sample_images/test.png' # example image  
image_dir = os.path.dirname(image_path)  
file_name = os.path.splitext(os.path.basename(image_path))[0]  
# output pathes  
obj_path = '/content/pifuhd/results/pifuhd_final/recon/result_%s_256.obj' % file_name  
out_img_path = '/content/pifuhd/results/pifuhd_final/recon/result_%s_256.png' % file_name
```

```
video_path = '/content/pifuhd/results/pifuhd_final/recon/result_%s_256.mp4' % file_name
video_display_path = '/content/pifuhd/results/pifuhd_final/result_%s_256_display.mp4' %
file_name
cd /content
```

Step-4

Preprocess (for cropping image)

```
!git clone https://github.com/Daniil-Osokin/lightweight-human-pose-estimation.pytorch.git
cd /content/lightweight-human-pose-estimation.pytorch/
!wget
https://download.01.org/opencv/opencvino\_training\_extensions/models/human\_pose\_estimation/ch
eckpoint\_iter\_370000.pth
import torch
import cv2
import numpy as np
from models.with_mobilenet import PoseEstimationWithMobileNet
from modules.keypoints import extract_keypoints, group_keypoints
from modules.load_state import load_state
from modules.pose import Pose, track_poses
import demo
def get_rect(net, images, height_size):
    net=net.eval()
    stride = 8
    upsample_ratio = 4
    num_keypoints= Pose.num_kpts
    previous_poses = []
    delay = 33
    for image in images:
        rect_path= image.replace('%s' % (image.split('.')[1]), '_rect.txt')
        img = cv2.imread(image, cv2.IMREAD_COLOR)
        orig_img = img.copy()
        orig_img= img.copy()
```

```

heatmaps, pafs, scale, pad = demo.infer_fast(net, img, height_size, stride, upsample_ratio, cpu=False)
total_keypoints_num = 0
all_keypoints_by_type = []
for kpt_idx in range(num_keypoints): # 19th for bg
    total_keypoints_num += extract_keypoints(heatmaps[:, :, kpt_idx], all_keypoints_by_type,
    total_keypoints_num)
pose_entries, all_keypoints = group_keypoints(all_keypoints_by_type, pafs)
for kpt_id in range(all_keypoints.shape[0]):
    all_keypoints[kpt_id, 0] = (all_keypoints[kpt_id, 0] * stride / upsample_ratio - pad[1]) /
    scale
    all_keypoints[kpt_id, 1] = (all_keypoints[kpt_id, 1] * stride / upsample_ratio - pad[0]) /
    scale
current_poses = []
rects = []
for n in range(len(pose_entries)):
    if len(pose_entries[n]) == 0:
        continue
    pose_keypoints = np.ones((num_keypoints, 2), dtype=np.int32) * -1
    valid_keypoints = []
    for kpt_id in range(num_keypoints):
        if pose_entries[n][kpt_id] != -1.0: # keypoint was found
            pose_keypoints[kpt_id, 0] = int(all_keypoints[int(pose_entries[n][kpt_id]), 0])
            pose_keypoints[kpt_id, 1] = int(all_keypoints[int(pose_entries[n][kpt_id]), 1])
            valid_keypoints.append([pose_keypoints[kpt_id, 0], pose_keypoints[kpt_id, 1]])
    valid_keypoints = np.array(valid_keypoints)
    if pose_entries[n][10] != -1.0 or pose_entries[n][13] != -1.0:
        pmin = valid_keypoints.min(0)
        pmax = valid_keypoints.max(0)
        center = (0.5 * (pmax[:2] + pmin[:2])).astype(np.int)
        radius = int(0.65 * max(pmax[0]-pmin[0], pmax[1]-pmin[1]))
    elif pose_entries[n][10] == -1.0 and pose_entries[n][13] == -1.0 and pose_entries[n][8] != -
    1.0 and pose_entries[n][11] != -1.0:
        # if leg is missing, use pelvis to get cropping

```

```
center = (0.5 * (pose_keypoints[8] + pose_keypoints[11])).astype(np.int)
radius = int(1.45*np.sqrt(((center[None,:] - valid_keypoints)**2).sum(1)).max(0))
center[1] += int(0.05*radius)
else:
center = np.array([img.shape[1]//2,img.shape[0]//2])
radius = max(img.shape[1]//2,img.shape[0]//2)
x1 = center[0] - radius
y1 = center[1] - radius
rects.append([x1, y1, 2*radius, 2*radius])
np.savetxt(rect_path, np.array(rects), fmt='%d')
net = PoseEstimationWithMobileNet()
checkpoint = torch.load('checkpoint_iter_370000.pth', map_location='cpu')
load_state(net, checkpoint)
get_rect(net.cuda(), [image_path], 512)
```

Step-5

Download the Pretrained Model

```
cd /content/pifuhd/
!sh ./scripts/download_trained_model.sh
```

Step-6

Run PIFuHD!

```
# Warning: all images with the corresponding rectangle files under -i will be processed.
!python -m apps.simple_test -r 256 --use_rect -i $image_dir
# seems that 256 is the maximum resolution that can fit into Google Colab.
# If you want to reconstruct a higher-resolution mesh, please try with your own machine.
```

Step-7

Render the Result

```
from lib.colab_util import generate_video_from_obj, set_renderer, video
renderer = set_renderer()
generate_video_from_obj(obj_path, out_img_path, video_path, renderer)
# we cannot play a mp4 video generated by cv2
!ffmpeg -i $video_path -vcodec libx264 $video_display_path -y -loglevel quiet
video(video_display_path)
```